Wombat

Wombat (design version 4) is designed to be a general purpose programming language which is capable of including mathematical statements and proofs.

Significant Expressions

Code can be at file level or in procedures. The main file is an expression, and it is expressions all the way down from there (with possible interspersed pragmas giving guidance to the compiler). An expression is an operator with some sub-expression arguments. Operators consist of suboperators with subexpressions interspersed. E.g. in the if-then-else operator: `if`, `then` and `else` are suboperators.

- … `=` … *(constrains both sides or fails)*
- … `:` … *(an expression with a constraining type)*
- `(` … `)` *(does nothing semantically)*
- … `,` … `,` … *(create a 3-tuple)*
- … `*` … `*` … *(create a 3-tuple type)*
- Note that `,` , `*` and others are repeating not binary operators. Prefix variant is the singular, so `(,3)` is a 1-tuple – explicitly a `Tuple[Nat]=(*Nat)`.
- Juxtaposition or space is a procedure call.
- Types behave as procedures that constrain the parameter but return themselves. So `T(t)` is the same as `(t:T;T)`. This is mostly used in procedure types and explicit closures. However it is not available for the Atom type: when a type gets an atom parameter it returns the named property.
- Space can be used as a suboperator in other operators, making its use for procedure calls inaccessible at such possible locations, however juxtaposition is always available for procedure calls.

A Note on Execution

Expressions can succeed or fail, or even remain unresolved. It is natural to assume that we want expressions to succeed, but that is wrong. We want the program to succeed. However success and failure are used for flow control, and it may be that, for the program to succeed it needs some subexpressions to fail. So at any point in execution we are targetting success or we are targetting failure or we don't care.

So consider the most common expression `x=y`. If we are targetting success (as we nearly always are) then this places constraints on both sides to be equal. If we are targetting failure then it constrains them to be not equal. If we don't care, because it leads to branches which both lead to program success, then it is just a check and doesn't constrain the `x` or `y`.

Code is easier to read if all expressions are targetting success. To make some contexts clear the `|` prefix operator to force success targetting. (Note that this is unrelated to the `|` infix operator which is procedure call with the operands reversed.)

Procedures, Functions, and Actions.

Procedures have one input type and one output type, and can be:

- Actions whose semantics are time dependent. I.e. impure, corresponding to effects in some languages. The type is written `X~>Y`, or `X~>>Y` if total (never fails);
- Functions are pure and execute outside time. The type is written `X=>Y`, or `X=>>Y` if total. These coerce to actions where required (with an empty collection of side effects).

There are **evil** variants of these 4, for code that peeks under the hood, allowing operations that don't respect the declared semantics. Evilness can be removed by proving that the declared semantics are respected. Prefix the arrow with □, e.g. `Int □=> Int`.

Embeddings of type `X` in type `Y` are a pair of procedures, with the `X=>>Y` procedure being total, while the reverse `Y=>X` will fail for `Y` values that do not come from an `X`. The type is written `X<=>>Y`. The two procedures are required to be inverses where either is defined. If `f` is a `X<=>>Y`, then `f.to` is the forward total procedure and `f.rev` is the reverse partial procedure. However `f` alone will convert to `f.to` with a non-Atom argument, and f^{-1} is always `f.rev`.

Closures

Non-primitive procedures are created with closures which have the form:

```
{ input-type arrow output-type : expression }
```

The arrow will be one of those from the various procedure type arrows, such as `=>`.

The input type can be "decorated" with identifiers which are set by the input argument, and which can be accessed in the closure's expression, but also in the output-type (dependent type). For example:

```
`f = { List(X)`lx =>> Vector(lx.len,X) : ... }
```

We can also decorate the output, in which case the result of the procedure is derived from relevant assignments and the end result of the expression is ignored, for example:

```
`f = { Int`x*Int`y=>>Int`z : z=x-y; debug.log "x=" x}; ...
```

If the output is not explicitly set (which means: reduced to an implied type with exactly one value) then the result of the expression is returned. If the output type is omitted then the inferred type of the expression is used. If the arrow is omitted then the compiler will give the best arrow it can infer, and the output type must also be missing. If the input type is missing then `Any` is assumed. Ths input value can be accessed as `$` and this is the only way when the input type is omitted from the head of the closure. The output value can be accessed as `` `$ ``. whose value will determine the closure's result.

If the arrow in the closure head is `<=>>` then that implies that the expression is capable of being run backwards and the closure then creates an embedding. One can write the code so that it also constructs the input from the output as well as in the forward direction.

Case and Combine

The `case` expression has the form `case` *expression* `of` *list-expression*, where the list is a list of procedures. For each of the procedures in the list where the expression has, or converts to, the input type of the expression, then it calls that procedure. If all calls fail then the `case` expression fails, otherwise the result of the `case` is the `combine` of the results. If all the elements of `y` are procedures then

```
case x of y = combine y x
```

In general the `combine` of a list is the common value if that can be determined (which is always the case if the type is `Distinguishable`), otherwise it checks if the type conforms to `Combinable` and applies the `combine` operation. For procedures that will be equivalent to the `case` operation, and if that returns procedures then they are combined. If we can determine that 2 elements in the list are not equal (which is also always possible for `Distinguishable` types), then the `combine` fails immediately.

`combine` is used in many place in wombat. [[FIXME: The original idea was that a combine of two procedures would, at execution time, give the one that succeeds or check that the two results agree (in the intersection of their types). However for the advanced uses of wombat we need to check this for all possible inputs.]]

There is also `firstCase` which is the way such switch statements work in other languages. If the procedures in the list are total, then `case` calls the ones whose domain match the parameter's type and try to `combine` the results. There is a `totalCase` variant where at least one domain must match and the `combine` is required to succeed. The proof is then likely to be easiest when the domains cover the possible inputs and don't overlap.

Types

A type has a defining collection of properties. It is either primitive, in which case it has an inbuilt implementation, or it (usually) has an implementation in terms of another type. Ultimately everything is implemented by primitive types and their properties. The properties of primitive types are axioms. The properties of other types are implemented using the properties of the implementing type.

Mathematicians call implementations "models". A model is a nice thing to have because without one you don't know if anything exists with those properties. However some things are useful to have because they are well behaved mathematically. An important example is the real numbers. We can only implement and deal with the computable reals, though we (perhaps) know that there are many more, so it seems that our implementations are incomplete.

Proposition Types

Types with at most 1 value are Proposition Types, with the type being regarded as a proposition, and the value, when present, being taken as a witness that there is a proof of the proposition. We say that they are of higher type `Prop` rather than `Type`. Lean4 will regards them as having a lower Universe level than `Type` but in wombat they are just a subtype of Type. Currently there is no Universe system.

A term is an expression that, directly or indirectly, includes free identifiers, typically because they are derived from the input type of a procedure.

Syntactically, the suboperators we use for `Prop`s are similar to those for the tests used for flow control, with the variation that: (a) text suboperators have the first letter capitalised; and (b) proposition suboperators have a percent (`%`) postfix. So:

- `x < y` returns its left argument (`x`) or fails;
- `x <? y` returns a boolean result;
- `x <% y` is a `Prop` type.
- `x @<% y` is the value of the corresponding `Prop` type, and is a claim that it has one.
- == is an alias for `=?` .

Set types

Types whose equality type is a `Prop` are `SetType` types (not to be confused with the `Set(X)` type). This will be all ordinary types. The type hierarchy makes these behave similarly to mathematical sets. For this initial version of wombat we ignore this subtle distinction and assume that all types are `SetType`s.

This is unrelated to the fact that values of type `Set(X)` coerce, where a type is required, to the type `X` restricted to those values.

W3 aspects part 1

- **Operators** are formed by a sequence of suboperators. In the above we have:
  - `{ … =>> … : … }` *(a closure for a total function)*
  - `< … : … >` *(create a structure type)*
  - `∀% …` *(create a `forall` proposition from a procedure)*
- **Identifiers** stand for a value of some sort (not mutable):
  - a backquote prefix means that this is new and different from an outer use of the same name.
  - a double backquote prefix indicates a new free identifier with an implicit "foreach". This is like a lower case type name in haskell, but it need not be a type. These can be reused in a scope. An explicit `foreach` creates new free identifiers without using backquotes.
  - backquotes are always at the start of identifiers or suboperators that they modify, and they break on the left, as we see above in `Type`T` and `T`u`. We note that backquotes at the start of non-identifier suboperators has no general meaning: it just creates a new suboperator.
  - syntactically, identifiers are operators with no left or right (nofix), and can make other suboperators inaccessible (in other words, suboperators which are identifiers are not reserved words).

Structure Types

There is a correspondence between the types of identifiers and the constraints imposed on identifiers by their being in an expression. Structure types utilise this. They have the form:

```
< parent-expression : expression >
```

The value of the expression is ignored. Rather the types of all the non-free identifiers introduced in the structure expression are visible using dot notation. A value of a structure types has values for all declared identifiers, meaning that the type then has only one value which can then be accessed with the `@` operator. Consider:

```
`AddTo3 = <: `x:Nat; `y:Nat; x+y=3;>;
`AddTo3x1 = <AddTo3: x=1;>; # AddTo3x1.y is the type Nat//{Nat`n:n=%2}
`AddTo3x1Value = @AddTo3x1; # Since there is only one value, we extract it.
AddTo3x1Value.y = 2 # succeeds if the compiler can work that out
```

However it is good form to specify all constraints by typed identifiers. Our example becomes:

```
`AddTo3 = <: `x:Nat; `y:Nat; `makes3: x+y =% 3;>;
```

The parent structure type will be above in the type hierarchy. So we can add new fields (which are properties) to those of the parent, or constrain existing ones.

Structure types are `Prop`s if all the fields are.

The fields in Structure types can depend on each other. This roughly correspond to dependent pairs in languages that support dependent types.

The Type Hierarchy

Every universe level has its own hierarchy. The rules for determining the universe level of an expression are not yet worked out but will probably use mugen, as narya intends to do. Currently it is possible to define the type of all types that are not members of themself, and prove both that it is and that it isn't a member of itself. A universe level system will fix this.

We use embedding pairs of functions, i.e. values of some `X<=>>Y` type, to define a type hierarchy resembling set inclusion. We start with some declarations that `X isa Y`, such as:

```
Int isa Rational by {Int`n<=>>Rational: n/1}  # can run backwards
```

This returns a value of the proposition type `Isa%(Int,Rational)`,

The lower type converts when required to the higher and that will always succeed. The higher type can be converted down to the lower when explicitly required, but that may fail. The lower type inherits all properties from the higher.

We extend this to a partial order by transitive closure, i.e. `X isa X`, and if `X isa Y` and `Y isa Z`, then `X isa Z`, using the obvious embeddings.

We then extend this to a lattice (a bounded order lattice) by adding LUBs (least upper bounds) and GLBs (greatest lower bounds) of any set of types. The intention is for these to make types behave similarly to sets in mathematics, so we name the LUB as `Union` and the GLB as `Intersection`.

If `S` is a set of types that only has finite `isa` chains, we apply the following rules:

- When `X isa Y` then `Union(X,Y)` is `Y` and `Intersection(X,Y)` is X. Applying this rule to pairs in `S` reduces it to an anti-chain.
- `Union(X,Union(Y,Z))` is `Union(X,Y,Z)`. Applying this rule eliminates `Unions` from our set `S`.
- Similarly `Intersection(X,Intersection(Y,Z))` is `Intersection(X,Y,Z)`.
- `Intersection(X,Union(Y,Z))` is converted to `Union(Intersection(X,Y),Intersection(X,Z))`.
- As a result, `Unions` can contain `Intersections` but not `Unions`, and `Intersections` can contain neither.

`Union` and `Intersection` (and infix operators `∩` and `∪` and the big ones that take a set input) are LUB and GLB ops for the lattice. `UnionType` and `IntersectionType` are the new types whose key property is an antichain of 2 or more types – in the case of `UnionType` it is also constrained to have no `UnionTypes`, and in the case of `IntersectionType` it is constrained to have neither `UnionTypes` nor `IntersectionTypes`.

For chains that are infinite upward we create a `Union` of the types in that chain where the values are those in any of the types and the properties are those in all of the types. For chains that are infinite downward we create an `Intersection` where the values are those in all the types and the properties are those in any of the types. Like all infinite things, infinite chains arise with relevant procedures, and can only be handled when relevant proofs about those procedures can be constructed. We gloss over the complexities of many infinite things in this document.

There is now a general rule for conversions within the hierarchy: When we need to convert an `X` to a `Y`, first convert the `X` value to `Intersection(X,Y)` (possibly failing), then convert from there to `Y`.

We now need to define what are the values and properties of these types.

The definition of `UnionType`

The values in a `UnionType` of a set of types `S` are the values from the separate types in `S`, together with a memory of which type thay are from. The programmer doesn't have direct access to that type information. Two values are equal in the `UnionType` if we can convert each to the `Intersection` of the types they come from and they are equal there. Two values are proven to be not equal if they are not equal in that `Intersection`, or if either of the downward conversions fail. While the situation can be much more complex, in the simplest non-trivial case, if `X isa Y` and `X isa Z` then two values in `UnionType(Y,Z)` will be equal iff they come from the same `X` value, irrespective of whether they are marked as a `Y` or a `Z`.

The `UnionType` is below all the types that are above every type in `S`. The properties of the `UnionType` are all inherited from those types above, plus the fallible conversion to a lower type. Note that the success of the downward conversion is independent of the specific lower type it is internally

associated with. So in our previous example if we convert from `Union(Y,Z)` to `Y`, that will succeed when the value has come up to the `UnionType` from `Z` if it is actually from `X` lower down.

The definition of `IntersectionType`

The properties of the `IntersectionType` are precisely the properties of any type in `S`.

The values of the `Intersection` are the `Union` of all the types that are below every type in `S`. In practical cases this recursive definition doesn't lead to any significant issues.

Important special cases

To complete our order lattice we create special types which are the top and bottom of the lattice. These arise when the parameter to the `Union` or `Intersection` is the empty set of types, ∅.

`Union(∅) = Empty`, the type with no values and all properties. It is at the bottom of the hierarchy with links up to all types, though, of course, it has no values to convert up, and all downward conversions to it will fail. So for any property identified by a Behaviour and an atom identifier, `Empty` has that property in the form of a `combine` of all the values of that property in any type that conforms to that Behaviour. This constrains all instances of a particular Behaviour to be somewhat compatible. The properties we deal with most are procedures that map to operators, and these are combinable if their inputs have empty intersection.

`Intersection(∅) = Any`, the type at the top of the hierarchy with all values and no properties, other than attempted conversions to lower types that are more useable.

The type lattice has many uses in wombat. Here's just one. The list `[3,"text"]` is a `List(Union(Nat,String))` (approximately). So the empty list `[]` is a `List(Union())` which is a `List(Empty)`. There is a lift of `isa` relations to `Lists` that is implemented like this:

```
foreach X:Type,Y:Type,xisay:Isa%(X,Y) do
  List(X) isa List(Y) by
    {List(X)`lx <=>> List(Y)`ly :
      ly = map({X`x=>>Y: x:Y}, lx);
      lx = map({Y`y=>X: y:X}, ly);
    }
od
```

Where the code constructs `ly` from `lx` and vice versa. But since `Empty` is below all other types, this converts `[]` to an empty list of any other type.

Subset types

The next aspect of the type hierarchy is subset types, which are similar to refinement types in some other languages. For type `T`, a subset type can be specified by `T//{T`t=>>Prop: proposition}`. The proposition is dependent on `t`, and `t` belongs to the subset type if the proposition is inhabited. This means that to determine that a value is in the subset we need to compute a proof, and similarly to determine that it is not in the subset we need a proof of the negation. These are trivial for a known value when the proposition corresponds to a computable boolean expression, but practical situations are likely to involve terms rather than values, or propositions containing universal qualifiers or both.

The `//` operator coerces its 2nd argument to be a total procedure from the type on the left to `Prop`. So we can always just write `{: expression}`, using `$` for the input.

Subset types inherit all their properties from their parent type, plus the dependent proposition.

Subset types make the type hierarchy vastly bigger, and introduce `isa` relations based on proposition implication.

Two important subset types are:
- A subset with a single value is the type of a successful expression evaluation. In particular a constant has such a type, so 42 has type `Nat//{:$=%42}`.
- A subset with no values is the type of a failed expression evaluation.

W3 Aspects, Part 2

The `foreach` construct allows us to, effectively, execute an infinite number of declaration operations. These can be `isa`, `conformsTo`, just procedure declarations, or other constraints.

When using a closure to define an embedding we need the code to work forward and backward to create the required pair of procedures. However the execution engine will try to run ordinary procedures backwards in the appropriate context, which is typically pattern matching. The programmer can make procedures more likely to successfully run backwards by giving the output a name and including code to explicitly calculate the input from the output. For example, here is a factorial program that will run backwards for input greater than 1:

```
`factorial =
  {Nat`n=>>Nat`fn:
    fn = case n of [{Nat 0: 1}, {Nat(`m+1): n*factorial(m)}];
    n = if fn>?1 then
      {: `f2n = {Nat`k*Nat`f=>Nat: if k==f then {:k} else {:f2n(k+1,(f/k)}};
      # note f/k is Rational, converting to Nat and might fail running backwards
      f2n(2,fn)  #
    } else {:n} # when n isn't supplied, this else won't advance its determination
  } # not an embedding because 0 and 1 both map to 1
```

In a total function, and indeed in any expression with no action calls, all subexpressions proceed together as soon as they can. So if we have `6=factorial(`x)` it won't be able to do the first line of `factorial`, but it can do the second part, which will give `n` a value, then do the 1st line which will act as a check. One consequence of this is that the arms of an `if` are passed as procedures, because otherwise they would both be evaluated.

Attempts to run non-embeddings backwards which fail are ignored – they don't directly cause expression failure.

Conversions extended

We've seen that if `X isa Y` then that generates automatic conversions between `X` and `Y`. This extends to a general conversion scheme between types in the hierarchy, where we convert the source to the `Intersection` of the source and destination, then convert up to the destination. This will fail iff that first down conversion fails. Of course it always fails for types that have an empty intersection.

There is a second conversion mechanism: `ConvertsTo`. This normally applies between types that have an empty intersection. Examples in the standard library include:
- `Tuples` convert to a list whose type is the `Union` of the tuple's elements' types.
- `Lists` convert to `MultiSets` (by forgetting the order).
- `Multisets` convert to `Sets` (by removing duplicates).
- `Sets` of `X` convert to the type `X` restricted to the values in the set.

We create these conversions with the `ConvertsTo` operator

```
X ConvertsTo Y by {X=>>Y: expression }
```

This creates a value of `Prop` type `ConvertsTo%(X,Y)`, which the compiler remembers in its logic database. Note that the conversion is just one way. We have the rule that `Isa%(X,Y) isa`

`ConvertsTo%(X,Y)`, forgetting the reverse conversion property. Naturally `ConvertsTo%` is symmetric and transitive, forming a partial order.

The rule for conflict is that, if there are multiple conversions defined between `X` and `Y`, then they have to agree, when more than one succeed. To be more exact: the procedures are combined (see the section on `case` and `combine`).

Equality

We come at equality from 2 directions. Firstly we recall that every type `T` has a total procedure `Eq%(T)` that takes 2 values of `T`, `x` and `y`, and returns a `Prop` type `Eq%(T)(x,y)`. A value (witness) of this type is the main definition of equality. There is, similarly, `Neq%(T)(x,y)` for inequality.

These propositions types can be extended by adding new definitions, but new definitions have to be compatible with previous ones, and, particularly and commonly with the initial version automatically generated from the implementation of type `T`. Extending a definition like this is the same as when we define a procedure with multiple constraints, for example:

```
`not:Bool=>>Bool; not true = false; not false = true; ...
```

There is a Sructure type named `Equivalence` for relations which are reflexive, symmetric and transitive. We then have a Structure `Distinguishable` which extends `Equivalence`. Types conforming to `Distinguishable` have a trio of operations such that defining any one of them will create the other 2. They are:

- A boolean function that gives true or false for any pair of values of the type. This is the == (or =?) operator.
- A function that takes two values of the type and returns the value if they are equal, otherwise fails. This is the = operator, though this operator is more often used to make expressions equal than to check them.
- A pair of procedures: one creates a witness for the `Eq% Prop` or fails, and the other creates a witness for the `Neq% Prop` or fails. One of these succeeds as they derive from a total procedure to the `Union` of the two `Prop`s.

Simple data types and all primitive types are `Distinguishable`. The main exceptions are procedures (and types which include or are defined in terms of procedures) and approximate numbers.

Types are where equality is defined, but as with many operations, it is natural to use the type hierarchy to extend the meaning in a sensible way. That commonly means seeing if the operation is meaningful in the `Union` of the types. So if we have `x:X` and `y:Y`, we would check the `Prop` `Eq%(Union(X,Y))(x,y)`. If we had `3:Int` and `5/2:Rational` then `Rational` is the `Union` and we would convert `3` to `3/1` and see from the `Rational` equality test that they are not equal. However there is a general definition of equality in a `Union` which says to take the values down to the `Intersection` of their associated types: if either fails to convert to that intersection then they are not equal, and otherwise we do the comparison there. In our example `5/2` will fail to convert to the `Intersection` (which is `Int`) so they can't be equal. If the `Rational` had been `3/1` then the conversion would succeed and the comparison would say they are `equal`. So rather than detour via the `Union`, we define the `=%` infix operator to take place in the `Intersection` of the argument's types. This carries over to the = and == operations.

Action

Actions are procedures with time-dependent semantics. A sophisticated system of typed effects might be desirable, but for the moment the emphasis is on the control of time.

Action calls have a start and end `TimeOrder`, which is a time plus a number which is such that `TimeOrders` of different action calls are always different. Placing constraints on these `TimeOrders` can allow close control of the order of execution. However in the main the control mechanism in effect is that action calls separated by semicolons are required to execute in left to right order.

Compilation and Execution

Compilation and execution are similar. The objective is to reduce the type hierarchy level of every expression. The compilation phase works from the outer scope inwards. Inner scopes can forward reference outer identifiers. This allows mutual recursion, however it is good form to mention identifiers before they are referenced in a lower scope, usually by specifying their type.

Compilation notes which expressions will influence other expressions as their type is increasingly constrained at run time, and arranges for appropriate updating messages to be sent.

At run time, every time an action completes, this advances every waiting expression.

Syntax

Nearly all syntax is operators. Operators map to an AST node, which is nearly always a procedure call. An operator consists of suboperators which are tokens from the code input. For example the if-then-else operator has suboperators: `if`, `then` and `else`. An operator is defined by a sequence of mandatory suboperators plus some slots for operands. If there are `n` mandatory suboperators then there are `n+1` slots: one to the left of the first and one after each. The operator is defined by those mandatory suboperators, plus the presence or absence of a required operand in each slot.

So if there is just one suboperator then there are four possible operators: infix with 2 operands; prefix with one operand; postfix with one operand; and nofix with no operands. Syntactically, identifiers are nofix operators

If two operands are adjacent then a space suboperator is inserted between them. In wombat by default this is used for procedure call. When the space suboperator is in use, as it normally is, then the last supoperator can't exist both with and without an operand. For example with just one suboperator, it can't have variants that are both infix and postfix, or both prefix and nofix. So an identifier declaration doesn't just block an existing nofix operator, but also a matching prefix operator.

After each mandatory operator there can also be a sequence of optional (0 or 1) and repeating (0 or more) suboperators. An example of this is the comma operator that creates tuples. The first comma is mandatory, but the operator then has another comma as a repeating suboperator.

The operands get combined into a tuple as follows:

- mandatory operands appear in the next slot in the tuple
- optional suboperators have a slot which is a 0-tuple `()` if absent and a 1-tuple if present. If the suboperator has no operand then the 1-tuple is `(())`.
- repeating arguments are a tuple of the operands, hence a 0-tuple if none.

The details of creating operators are (will be) given in an appendix.

Builtin Types

xxx

Notes

Note on Equality

0. Equality is (`conformsTo`) an `Equivalence` relation.
1. Primitive types are `Distinguishable`, so any 2 values we can say if they are equal or not. Typically this is just a bit pattern match.
2. Builtin types aren't necessarily primitive, but they will have a definition of equality, which may allow one to prove that 2 values are, or are not, equal. e.g. total procedures are equal if equal inputs always give equal outputs.
3. All types have an implementation in terms of some other type, and ultimately everything gets back to primitive types (which might be builtin) or to other builtin types which will at least have a mechanism for proving equality. Basically/probably all builtin types that aren't primitive will be related to procedures. For example the `Real` numbers might be defined by a procedure that lets you get arbitrarily close to it without leaving the `Rationals`.
4. All types have a notion of equality. However by default 2 values are equal if they have equal implementations, and there is no default way to prove that two values are unequal. This means that if you define a new value by `x=y` and then later ask if `x` equals `y`, then the answer will be yes. This is `BaseEquality`.
5. As for procedures, we can define Equality and Inequality by declaring that the defining extends one or both of `Eq%` and `Neq%` behaviours. For simple types we declare that it extends `Distinguishable` which allows (in)equality boolean to be computed.
6. Equality test happens in the `Intersection` of the 2 types.

Note on Execution

0. Compilation time is just the xecution that can be performed before time 0.
1. We start with the intention to make the program succeed. However we use failure for flow control. It may be that we need some subexpression to fail to make its parent succeed. So sometimes we are looking for the constraints that make some subexpression fail. Failed expressions are likely to cause parents to fail but propogation only happens if the whole expression of a procedure fails.
2. During execution every (sub)expression has a type. Execution moves that type down the hierarchy. If it reaches a type restricted to a single value then that is a successful evaluation. If it reaches a type restricted to the empty set that says that execution has failed. When we call a procedure we pass in what we know about the input and what we know about the output, and the procedure will sometimes improve one or the other, and that change will hopefully cause a cascade of other changes. Even if the type has got down to a single value, that might not be the end if that value is inconsistent with some constraint, moving the expression down to failure, A start on this is in /sw/marsupial and the current code in this wombat is just a mod of that.
3. ; operator discards the left operand and returns the right. However failure of the left causes the whole to fail.
4. Getting the type hierarchy to work is crucial. It commonly involves proofs being given by the user or (preferably) discovered by the compiler.